Machine Learning Laboratory
Experiment 05
Implementation of Classification Algorithms

Algorithms used: Logistic Regression, K-Nearest Neighbours (KNN), and Gaussian Naive Bayes.

Focus: The theory of these three algorithms has already been covered in class. This experiment focuses on preparing a raw dataset correctly and then applying the classifiers.

1 Aim

To perform data inspection, data cleaning, preprocessing, feature scaling, and binary classification on the Pima Indians Diabetes dataset using Logistic Regression, KNN, and Gaussian Naive Bayes, and compare their performance using accuracy.

2 Objectives

1. Load a raw CSV dataset using Pandas.
2. Inspect its structure, data types, duplicates, missing values, and suspicious values.
3. Identify zero values that represent missing information in selected columns.
4. Convert invalid zeros into missing values and handle them using median imputation.
5. Inspect numerical features for possible outliers.
6. Separate input features and target.
7. Split the data into training and testing sets.
8. Standardize features without using the test set to learn scaling parameters.
9. Train Logistic Regression, KNN, and Gaussian Naive Bayes.
10. Predict the test classes and calculate accuracy.
11. Compare the three classifiers.

3 Dataset

The **Pima Indians Diabetes Dataset** is a binary classification dataset containing 768 observations and 8 input features plus a target variable.

Column	Type	Description
`Pregnancies`	Numerical	Number of pregnancies
`Glucose`	Numerical	Plasma glucose concentration
`BloodPressure`	Numerical	Diastolic blood pressure
`SkinThickness`	Numerical	Triceps skin fold thickness
`Insulin`	Numerical	Two-hour serum insulin level
`BMI`	Numerical	Body Mass Index
`DiabetesPedigreeFunction`	Numerical	Diabetes pedigree function value
`Age`	Numerical	Age
`Outcome`	Target	0 = negative class, 1 = positive class

The dataset is used for educational machine learning purposes. Students are not required to make medical diagnoses or interpret clinical significance.

4 Required Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score
```

5 Step 1: Load the Raw CSV

The raw file commonly contains no header row. Therefore, column names must be supplied explicitly.

Save the raw file as `pima-indians-diabetes.data.csv` in the same folder as the notebook.

```
columns = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age",
    "Outcome"
]

data = pd.read_csv(
    "pima-indians-diabetes.data.csv",
    header=None,
    names=columns
)

print(data.head())
```

Why `header=None`?

It tells Pandas that the CSV does not contain a header row. The `names=` argument then assigns meaningful column names.

6 Step 2: Inspect the Dataset

6.1 2.1 First five rows

```
print(data.head())
```

6.2 2.2 Last five rows

```
print(data.tail())
```

6.3 2.3 Number of rows and columns

```
print(data.shape)
```

Expected shape:

$$768 \times 9$$

6.4 2.4 Column names

```
print(data.columns)
```

6.5 2.5 Data types

```
print(data.dtypes)
```

6.6 2.6 Statistical summary

```
print(data.describe())
```

> A numerical value is not automatically a valid value. Data inspection must consider the meaning of the values, not only their data type.

7 Step 3: Check Duplicate Records

```
print("Number of duplicate rows:",
      data.duplicated().sum())
```

If duplicates are present, remove them:

```
data = data.drop_duplicates()
print("Shape after removing duplicates:", data.shape)
```

Record the number of duplicates in the observation table.

8 Step 4: Check for Actual Missing Values

```
print(data.isnull().sum())
```

The raw file may show zero missing values according to `isnull()`. However, this dataset contains another data-quality issue: some zero values are used where a physiological measurement would not normally be zero.

9 Step 5: Identify Invalid Zero Values

First count zeros in every column:

```
print((data == 0).sum())
```

For this dataset, treat zero as missing information in:

- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI

Do **not** replace every zero.

Valid zeros: `Outcome=0` is a valid class label. `Pregnancies=0` is also a valid observation. Therefore, only the selected measurement columns should be cleaned.

10 Step 6: Convert Invalid Zeros to NaN

```
invalid_zero_columns = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]

data[invalid_zero_columns] = data[
    invalid_zero_columns
].replace(0, np.nan)

print(data.isnull().sum())
```

The cleaning operation is:

$$\text{Invalid zero} \longrightarrow \text{Missing value (NaN)}$$

11 Step 7: Measure Missing Values

11.1 7.1 Count missing values

```
missing_values = data.isnull().sum()
print(missing_values)
```

11.2 7.2 Calculate missing percentage

```
missing_percentage = (
    data.isnull().sum() / len(data)
) * 100

print(missing_percentage)
```

$$\text{Missing Percentage} = \frac{\text{Number of Missing Values}}{\text{Total Records}} \times 100$$

This gives a better idea of the extent of missing information.

12 Step 8: Handle Missing Values

For this introductory experiment, use **median imputation** for the numerical measurement columns.

```
for column in invalid_zero_columns:
    data[column] = data[column].fillna(
        data[column].median()
    )

print(data.isnull().sum())
```

12.1 Why median?

The median is less affected by extreme observations than the mean. It is therefore a simple choice for numerical data that may contain outliers.

Note on machine learning practice: In a production-quality pipeline, imputation statistics should be learned from the training data only. This avoids data leakage. Here, the purpose is first to make the cleaning and preprocessing concepts clear; the leakage-safe pipeline approach can be introduced later.

13 Step 9: Verify the Cleaned Data

```
print("Missing values:")
print(data.isnull().sum())

print("\nCleaned data:")
print(data.head())

print("\nUpdated statistics:")
print(data.describe())
```

The selected measurement columns should now contain no missing values.

14 Step 10: Check for Possible Outliers

A boxplot can be used to inspect numerical features.

```python
import matplotlib.pyplot as plt

data[invalid_zero_columns].boxplot(figsize=(12, 6))
plt.xticks(rotation=45)
plt.title("Boxplots of Selected Features")
plt.tight_layout()
plt.show()
```

Do not automatically delete every outlier. An extreme value may be a genuine observation. In this introductory laboratory, identify and report possible outliers rather than automatically removing them.

15 Step 11: Separate Features and Target

The target column is `Outcome`.

```python
X = data.drop("Outcome", axis=1)
y = data["Outcome"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

Thus:

$$X = \text{input features}, \qquad y = \text{target}$$

The target is binary:

$$y \in \{0, 1\}$$

16 Step 12: Check Class Distribution

```python
print(y.value_counts())
print(y.value_counts(normalize=True) * 100)
```

This shows how many observations belong to each class and their percentages.

17 Step 13: Train-Test Split

Use an 80:20 split.

```python
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

`stratify=y`: attempts to maintain a similar class distribution in the training and testing sets.

`random_state=42`: makes the split reproducible.

18 Step 14: Feature Scaling

The features have different numerical ranges. KNN is particularly sensitive to feature scale because it uses distances. Standardization is also commonly useful for Logistic Regression.

Use:

$$z = \frac{x - \mu}{\sigma}$$

where x is the original value, μ is the training mean, and σ is the training standard deviation.

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Avoid data leakage:
Use `fit_transform()` on training data.
Use only `transform()` on testing data.
Do not fit the scaler using the test set.

19 Step 15: Logistic Regression

19.1 Train the model

```
logistic_model = LogisticRegression(max_iter=1000)

logistic_model.fit(
    X_train_scaled,
    y_train
)
```

19.2 Predict

```
y_pred_lr = logistic_model.predict(
    X_test_scaled
)
```

19.3 Calculate accuracy

```
accuracy_lr = accuracy_score(
    y_test,
    y_pred_lr
)

print("Logistic Regression Accuracy:",
      accuracy_lr * 100, "%")
```

20 Step 16: K-Nearest Neighbours

Use $k = 5$ initially.

```
knn_model = KNeighborsClassifier(
    n_neighbors=5
)

knn_model.fit(
    X_train_scaled,
    y_train
)

y_pred_knn = knn_model.predict(
    X_test_scaled
)

accuracy_knn = accuracy_score(
    y_test,
    y_pred_knn
)

print("KNN Accuracy:",
      accuracy_knn * 100, "%")
```

Student task: Repeat KNN for $k = 3$, $k = 7$, and $k = 9$. Record the accuracy for each value.

21 Step 17: Gaussian Naive Bayes

Because the input features are numerical, use Gaussian Naive Bayes.

```
nb_model = GaussianNB()

nb_model.fit(
    X_train_scaled,
    y_train
)

y_pred_nb = nb_model.predict(
    X_test_scaled
)

accuracy_nb = accuracy_score(
    y_test,
    y_pred_nb
)

print("Naive Bayes Accuracy:",
      accuracy_nb * 100, "%")
```

22 Step 18: Compare the Classifiers

```
results = pd.DataFrame({
    "Algorithm": [
```

```
        "Logistic Regression",
        "KNN",
        "Naive Bayes"
    ],
    "Accuracy (%)": [
        accuracy_lr * 100,
        accuracy_knn * 100,
        accuracy_nb * 100
    ]
})

print(results)
```

Algorithm	Test Accuracy (%)
Logistic Regression	
KNN ($k = 5$)	
Gaussian Naive Bayes	

Do not write fixed accuracy values in the laboratory record before running the program. Record the values obtained from your execution.

23 Step 19: Identify the Highest Accuracy

```
best_algorithm = results.loc[
    results["Accuracy (%)"].idxmax()
]

print("Best algorithm based on accuracy:")
print(best_algorithm)
```

For this experiment, the classifier with the highest test accuracy is considered the best performer **according to accuracy only**.

24 Accuracy

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} \times 100$$

For this laboratory, only accuracy is used. Precision, recall, F1-score, and confusion matrix will be studied separately.

25 Complete Program

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
```

```
10
11 from sklearn.metrics import accuracy_score
12
13
14 # 1. Column names
15 columns = [
16     "Pregnancies",
17     "Glucose",
18     "BloodPressure",
19     "SkinThickness",
20     "Insulin",
21     "BMI",
22     "DiabetesPedigreeFunction",
23     "Age",
24     "Outcome"
25 ]
26
27
28 # 2. Load raw CSV
29 data = pd.read_csv(
30     "pima-indians-diabetes.data.csv",
31     header=None,
32     names=columns
33 )
34
35
36 # 3. Inspect
37 print("Shape:", data.shape)
38 print("\nData types:")
39 print(data.dtypes)
40 print("\nSummary:")
41 print(data.describe())
42 print("\nDuplicate rows:",
43       data.duplicated().sum())
44
45
46 # 4. Remove duplicates
47 data = data.drop_duplicates()
48
49
50 # 5. Invalid zero columns
51 invalid_zero_columns = [
52     "Glucose",
53     "BloodPressure",
54     "SkinThickness",
55     "Insulin",
56     "BMI"
57 ]
58
59 print("\nZero counts before cleaning:")
60 print((data == 0).sum())
61
62
63 # 6. Convert invalid zeros to NaN
64 data[invalid_zero_columns] = data[
65     invalid_zero_columns
66 ].replace(0, np.nan)
67
```

```
68
69  # 7. Missing values
70  print("\nMissing values before imputation:")
71  print(data.isnull().sum())
72
73
74  # 8. Median imputation
75  for column in invalid_zero_columns:
76      data[column] = data[column].fillna(
77          data[column].median()
78      )
79
80
81  # 9. Verify cleaning
82  print("\nMissing values after imputation:")
83  print(data.isnull().sum())
84
85
86  # 10. Features and target
87  X = data.drop("Outcome", axis=1)
88  y = data["Outcome"]
89
90
91  # 11. Train-test split
92  X_train, X_test, y_train, y_test = train_test_split(
93      X,
94      y,
95      test_size=0.20,
96      random_state=42,
97      stratify=y
98  )
99
100
101 # 12. Scaling
102 scaler = StandardScaler()
103
104 X_train_scaled = scaler.fit_transform(X_train)
105 X_test_scaled = scaler.transform(X_test)
106
107
108 # 13. Logistic Regression
109 logistic_model = LogisticRegression(max_iter=1000)
110
111 logistic_model.fit(
112     X_train_scaled,
113     y_train
114 )
115
116 y_pred_lr = logistic_model.predict(
117     X_test_scaled
118 )
119
120 accuracy_lr = accuracy_score(
121     y_test,
122     y_pred_lr
123 )
124
125
```

```
# 14. KNN
knn_model = KNeighborsClassifier(
    n_neighbors=5
)

knn_model.fit(
    X_train_scaled,
    y_train
)

y_pred_knn = knn_model.predict(
    X_test_scaled
)

accuracy_knn = accuracy_score(
    y_test,
    y_pred_knn
)


# 15. Gaussian Naive Bayes
nb_model = GaussianNB()

nb_model.fit(
    X_train_scaled,
    y_train
)

y_pred_nb = nb_model.predict(
    X_test_scaled
)

accuracy_nb = accuracy_score(
    y_test,
    y_pred_nb
)


# 16. Comparison
results = pd.DataFrame({
    "Algorithm": [
        "Logistic Regression",
        "KNN",
        "Naive Bayes"
    ],
    "Accuracy (%)": [
        accuracy_lr * 100,
        accuracy_knn * 100,
        accuracy_nb * 100
    ]
})

print("\nFinal Results:")
print(results)
```

26 Expected Workflow

Raw CSV → Inspect → Check Duplicates → Check Missing Values → Identify Invalid Zeros → Convert to NaN → Impute Missing Values → Check Outliers → Separate X and y → Train/Test Split → Scaling → Train Classifiers → Predict → Accuracy → Compare

27 Observation Table

Observation	Value
Number of records before cleaning	
Number of columns	
Number of duplicate rows	
Invalid zeros in Glucose	
Invalid zeros in BloodPressure	
Invalid zeros in SkinThickness	
Invalid zeros in Insulin	
Invalid zeros in BMI	
Missing values after zero conversion	
Missing values after imputation	
Training set size	
Testing set size	

28 Results Table

Algorithm	Accuracy (%)
Logistic Regression	
KNN ($k = 5$)	
Gaussian Naive Bayes	

29 Student Exercise

Repeat the KNN experiment with:

$$k = 3, \quad k = 5, \quad k = 7, \quad k = 9$$

Record:

k value	Accuracy (%)
3	
5	
7	
9	

30 Conclusion

The raw Pima Indians Diabetes CSV dataset was inspected and cleaned before classification. Invalid zero values in selected measurement columns were treated as missing information and

handled using median imputation. Possible outliers were inspected without automatically deleting them. The data was then separated into features and target, divided into training and testing sets, and standardized. Logistic Regression, KNN, and Gaussian Naive Bayes were trained and evaluated using test accuracy. Their performance was compared based on the accuracy obtained from the experiment.

31 Post-Lab Questions

1. Why should a dataset be inspected before model training?
2. Why does `isnull().sum()` alone not identify all missing information in this dataset?
3. Why should not every zero in the dataset be replaced with `NaN`?
4. Why is `Outcome=0` valid?
5. Why is `Pregnancies=0` valid?
6. Why can zero in the Glucose column be treated as missing information?
7. What is the difference between an actual `NaN` and an invalid zero?
8. Why is median imputation used?
9. Why are duplicate records checked?
10. Why should outliers not automatically be removed?
11. Why do we split data into training and testing sets?
12. What is the purpose of `stratify=y`?
13. Why is feature scaling particularly important for KNN?
14. What is the difference between `fit_transform()` and `transform()`?
15. What is data leakage?
16. Why should the scaler not be fitted using the test set?
17. What does $k = 5$ mean in KNN?
18. Which classifier obtained the highest accuracy?
19. Does the highest accuracy mean that an algorithm is always the best classifier? Explain.

Key learning point:
Machine learning does not begin with selecting an algorithm. A reliable workflow begins with understanding, cleaning, and preparing the data, followed by training, prediction, and evaluation.